

Hospital Management System - Project Progress Report

1. Project Goal

The objective of this project is to build a Hospital Management System using Spring Boot, MySQL, and Postman. The system manages patients, doctors, and later appointments. It provides REST APIs for storing and retrieving hospital data.

2. Technologies Used

Java 17, Spring Boot, Spring Data JPA, Hibernate, MySQL Database, Maven, Postman, VS Code.

3. Project Structure

Entity Layer: Patient, Doctor. Repository Layer: JpaRepository interfaces. Service Layer: Business logic. Controller Layer: REST API endpoints. Database: MySQL hospital_db.

4. What We Completed

Configured Spring Boot application. Connected MySQL database. Created Patient entity. Generated patients table. Created Patient Repository, Service, Controller. Tested POST and GET APIs in Postman. Successfully inserted and retrieved patient records. Created Doctor entity. Created Doctor Repository, Service, Controller. Verified doctor records through GET endpoint.

5. Flow of Request Processing

Postman sends HTTP request -> Controller receives request -> Service processes request -> Repository interacts with database through JPA -> Hibernate generates SQL -> MySQL stores/retrieves data -> Response returned as JSON.

6. Patient API Details

POST /patients saves a patient. GET /patients retrieves all patients. Example fields: patientId, patientName, age, gender, phone.

7. Doctor API Details

POST /doctors saves a doctor. GET /doctors retrieves all doctors. Example fields: doctorId, doctorName, specialization, experience, phone.

8. Problems Encountered

Initial database schema issue caused auto_increment conflict. Resolved by recreating table structure. Postman Cloud Agent could not access localhost. Switched to Desktop Agent. Encountered HTTP 400 errors while testing doctor API and performed debugging.

9. Questions Mentor May Ask

What is Spring Boot? Why use JPA? What is Hibernate? What is @Entity? What is JpaRepository? Difference between Controller, Service, Repository? Why use @RequestBody? What is REST API? How does Hibernate create tables? Why use MySQL?

10. Answers Summary

Spring Boot simplifies Java web application development. JPA maps Java objects to database tables. Hibernate is the JPA implementation. @Entity maps a class to a table. JpaRepository provides CRUD operations. Controller handles requests. Service contains business logic. Repository communicates with database. @RequestBody converts JSON to Java object.

11. Database Tables

patients(patient_id, patient_name, age, gender, phone) doctors(doctor_id, doctor_name, specialization, experience, phone)

12. Current Status

Patient module completed and tested. Doctor module created and largely functional. Application starts successfully and connects to MySQL. REST APIs available on localhost:8080.

13. Next Steps

Create Appointment entity. Create Appointment Repository, Service, Controller. Connect Patient and Doctor with Appointment. Add Update and Delete APIs. Add validation. Build React frontend. Implement authentication and authorization.